多変量解析及び大規模計算論

（前半）玉森聡：多変量解析
（後半）内種岳詞：大規模計算論

ガイダンス

- はじめに（自己紹介・メッセージ）
- シラバス確認（15回の授業計画と評価基準）
- 準備・宿題（来週までにPCをセットアップ）
- まとめ

はじめに

- 自己紹介
  - 氏名：　内種 岳詞（うちたね たけし）
  - 専門：　応用数理，大規模計算，社会科学

《サークル・行事・花・風景》　日記より
http://ishikenf.at.webry.info/200607/article_20.h

$$
\begin{aligned}
dx_i = &\quad [v_i + \rho\xi_i]dt + \sigma dw_i. \\
dv_i = \{ &\quad -\alpha \sum_{j=1, j\neq i}^{N} \left[\frac{1}{(||x_i - x_j||/r)^p} - \frac{1}{(||x_i - x_j||/r)^q} \right] (x_i - x_j) \\
&\quad -\beta \sum_{j=1, j\neq i}^{N} \left[\frac{1}{(||x_i - x_j||/r)^m} + \frac{1}{(||x_i - x_j||/r)^q} \right] (v_i - v_j)\} dt.
\end{aligned}
$$

メッセージ(後半担当：内種)

- 大規模計算とは
  - いわゆる並列計算(CPU内，CPU間，PC間)
- 大規模計算の課題
  - 課題に合った工夫が必要
  - 効率的かつ効果的な方法で解決する能力
  - 問題全体を俯瞰する能力
- 目指すところ
  - 皆さん自身の研究課題を効率化し効果的な結果創出

担当回での評価方法

- ９回１０回は座学←出席取ります
  - 大規模計算とは
  - 大規模計算の課題解決法
- １１回１２回は演習←プログラムやレポートを評価します
  - 並列処理(による確率分布の実現値近似)
  - 分散処理(のための進化計算・ベイズ最適化・粒子フィルタ)
- １３回～１５回プレゼン←発表・質疑応答を評価します
  - 各自1回発表：持ち時間10分(発表8分・議論2分)
  - 毎回8名発表

$Bayes$(ベイズ）の定理

・ $Bayes$(ベイズ）の定理：

全事象$\Omega = \mathrm{A}_1 \cup \mathrm{A}_2 \cup \cdot\cdot\cdot\cdot A_n$ （但し $A_i \cap A_j = \Phi \quad (i \neq j)$),

事象A_iの起こる確率が$P(A_i)$であるとき，任意の事象Bに対して,

$$\boxed{P(A_i|B) = \frac{P(A_i)P(B|A_i)}{\sum_{k=1}^{n} P(A_k)P(B|A_k)}} \quad \cdot\cdot\cdot\cdot \quad 式(※)$$

式(B)の証明：

式(B)の分子 $P(A_i)P(B|A_i) = P(A_i \cap B)$

式(B)の分母 $$\sum_{k=1}^{n} P(A_k)P(B|A_k) = \sum_{k=1}^{n} P(A_k \cap B) = P(B)$$

であるから，式(※)は， $P(A_i|B) = \dfrac{P(A_i \cap B)}{P(B)}$ と書ける

即ち $P(B)P(A_i|B) = P(A_i \cap B)$（確率の乗法定理) (証明終）

周辺化の積分が計算できない

- **ベイズの定理の「壁」**
  - 周辺化積分が計算不能
  - $p(D) = \int p\,(D \mid \theta) p(\theta) d\theta$
  - θが高次元のとき指数的爆発
  - 正規分布なら解けるが一般には無理

- **分布の実現値近似（Monte Carlo）**
  - 粒子近似
  - $E[f(\theta)] \approx \frac{1}{N} \sum_{i=1}^{N} f\left(\theta^{(i)}\right)$

粒子近似したらできること

- 進化計算
  - パラメータ最適化
  - ブラックボックス関数に対応
- ベイズ最適化
  - パラメータ最適化
  - 予測モデルも同時に構築
- 粒子フィルタ
  - フィルタとは信号からノイズを除去

準備

- Google colab（googleアカウントが必要）でプログラムを実行

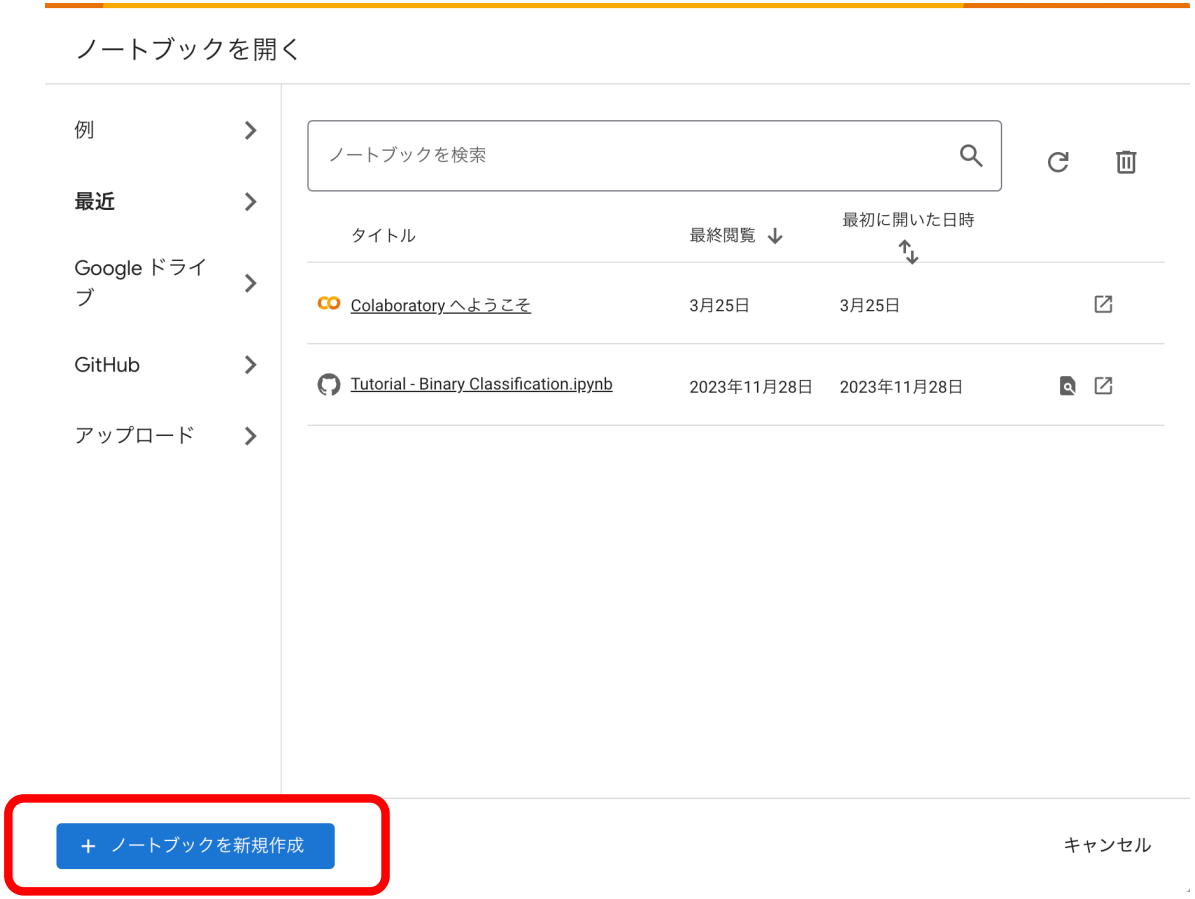
```
import numpy as np
import time

arr = np.ones((10000,10000))

def npdot_time(arr):
  start = time.time()
  np.dot(arr, arr)
  return time.time() - start

npdot_time(arr)
```

備考：
Moodleからダウンロード可
MA_HPC_01.ipynb
VS codeで開けます

解説：
float64型実数を1万個掛け算
実行時間を見てみましょう

手順 1

- ログインして**ノートブックを新規作成**



手順２

- プログラムをコピペして実行(⌘+return)

```
[5] import numpy as np
    import time

    arr = np.ones((10000,10000))

    def npdot_time(arr):
      start = time.time()
      np.dot(arr, arr)
      return time.time() - start

    npdot_time(arr)
```

```
60.04939532279968
```

コーディングを開始するか、AI で生成します。

手順 3

- 保存「MA_HPC01.ipynb」
  - “MA_HPC”

    = Multivariate Analysis and

    High Performance Computing

- “ipynb”

  = Ipython NoteBook形式

  中身はJSON

  - VS Codeで読めます

①名前を変更

①ipynbで保存

(option)個人PCでpython設定

- Windowsユーザー
  - ダウンロードサイト（https://www.python.org/downloads/windows/）
- Macユーザー
  - Home brew使っている人はpythonインストール済み
- Poser Shell(Win) or Terminal(Mac)でモジュール追加＋jupyter実行
  - `python -m pip install numpy japanize-matplotlib pandas scikit-learn jupyter`
  - `python –m jupyter notebook` ←ブラウザが起動します．終了はCtrl+C

（注）研究でpython使う人は環境分けるためgoogle colab利用を推奨

まとめ

- 内種の担当は第9回からです
- 座学：9回,10回（アクティブ・ラーニングにするかも）
- 演習：11回,12回（google colab使います）
  - 個人PCにpython+jupyterをセットアップしてもOK
- プレゼンテーション：13回,14回,15回(発表10分・議論10分)